

Lesson 02 Demo 02

Building a Financial Q&A Agent Using LangChain

Objective: To build a LangChain and Azure OpenAI–based financial Q&A agent that interprets business questions, generates and runs Python code, and explains results in plain English to speed up analysis and reduce manual errors

Tools required: Python, LangChain, Azure OpenAI API, and Visual Studio Code

Prerequisites: Familiarity with LangChain agents and tools

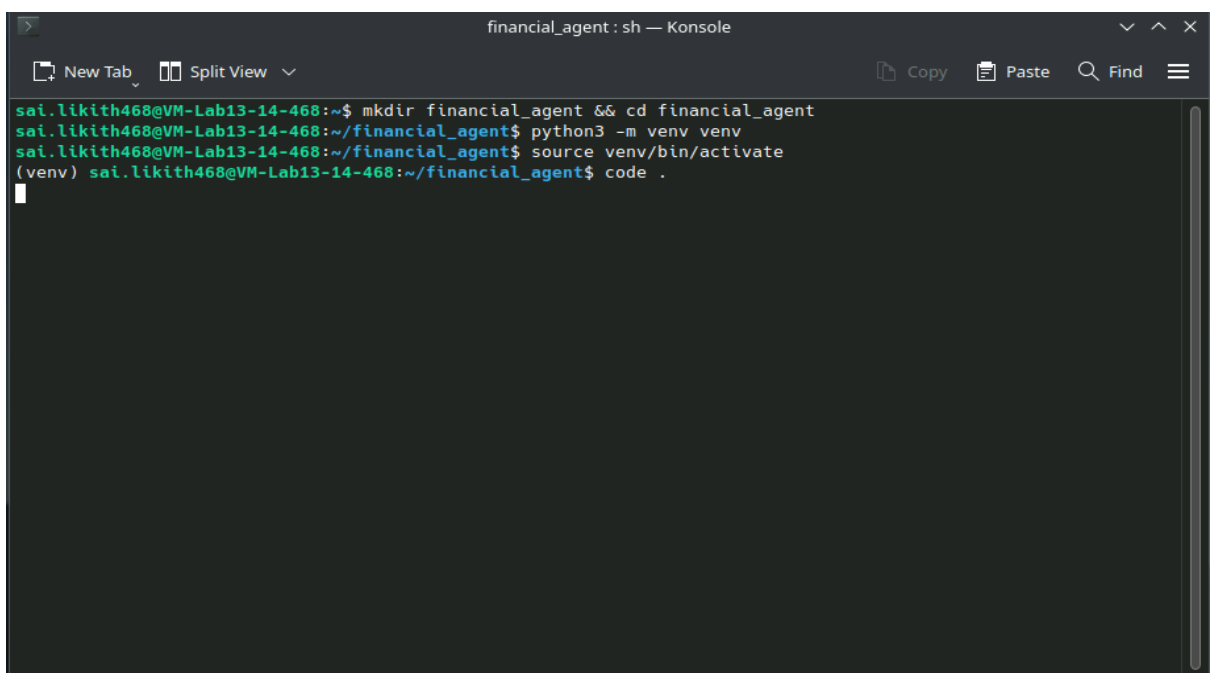
Steps to be followed:

1. Set up your local environment in Visual Studio Code
2. Configure the Azure OpenAI model
3. Define the execution tool and prompt template
4. Define the processing logic and business questions
5. Run the agent loop and view the outputs

Step 1: Set up your local environment in Visual Studio Code

1.1 Open the terminal in your system and run the following commands:

```
mkdir financial_agent && cd financial_agent
python3 -m venv venv
source venv/bin/activate
code .
```



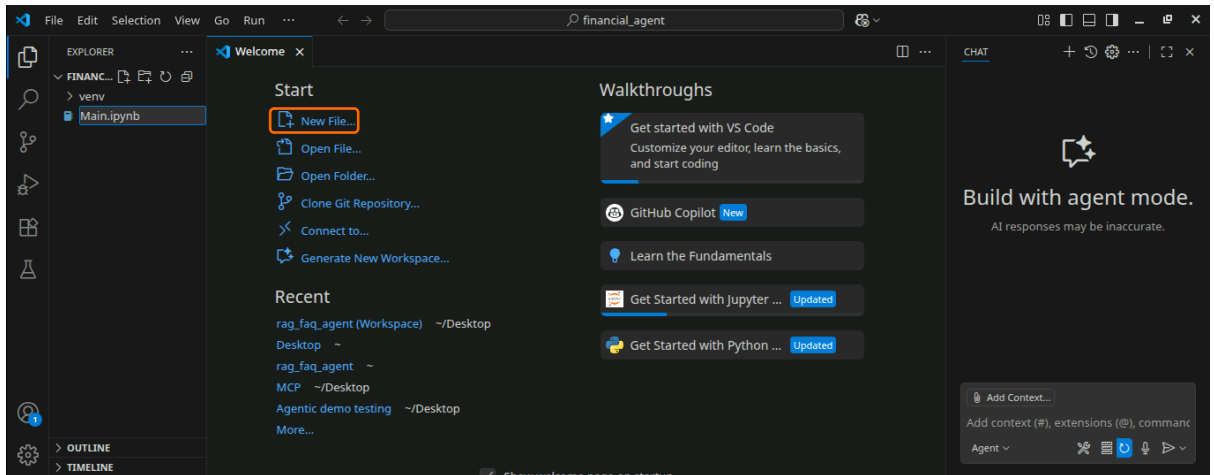
The screenshot shows a terminal window titled "financial_agent : sh — Konsole". The terminal displays the following commands and their output:

```
sai.likith468@VM-Lab13-14-468:~$ mkdir financial_agent && cd financial_agent
sai.likith468@VM-Lab13-14-468:~/financial_agent$ python3 -m venv venv
sai.likith468@VM-Lab13-14-468:~/financial_agent$ source venv/bin/activate
(venv) sai.likith468@VM-Lab13-14-468:~/financial_agent$ code .
```

The terminal window has a dark theme and includes standard window controls (minimize, maximize, close) and a search bar at the top right.

Note: This creates a virtual environment inside the `financial_agent` folder and opens the project in VS Code.

1.2 Click **New File...** to create a new file, and name it **main.ipynb**



1.3 In the first cell in the file, run the following command to install required packages:

!pip install langchain langchain-openai langchain-community

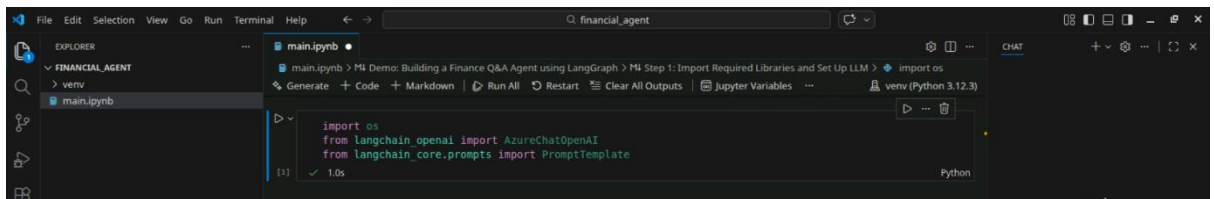
```
!pip install langchain langchain-openai langchain-community

Requirement already satisfied: langchain in ./venv/lib/python3.12/site-packages (0.3.27)
Requirement already satisfied: langchain-openai in ./venv/lib/python3.12/site-packages (0.3.30)
Collecting langchain-community
  Using cached langchain_community-0.3.27-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: langchain-core<1.0.0,>=0.3.72 in ./venv/lib/python3.12/site-packages (from langchain) (0.3.74)
Requirement already satisfied: langchain-text-splitters<1.0.0,>=0.3.9 in ./venv/lib/python3.12/site-packages (from langchain) (0.4.14)
Requirement already satisfied: langsmith<=0.1.17 in ./venv/lib/python3.12/site-packages (from langchain) (0.4.14)
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in ./venv/lib/python3.12/site-packages (from langchain) (2.11.7)
Requirement already satisfied: SQLAlchemy<3,>=1.4 in ./venv/lib/python3.12/site-packages (from langchain) (2.0.43)
Requirement already satisfied: requests<3,>=2 in ./venv/lib/python3.12/site-packages (from langchain) (2.32.4)
Requirement already satisfied: PyYAML<=5.3 in ./venv/lib/python3.12/site-packages (from langchain) (6.0.2)
Requirement already satisfied: openai<2.0.0,>=1.99.9 in ./venv/lib/python3.12/site-packages (from langchain-openai) (1.99.9)
Requirement already satisfied: tiktoken<1,>=0.7 in ./venv/lib/python3.12/site-packages (from langchain-openai) (0.11.0)
Collecting aiohttp<4.0.0,>=3.8.3 (from langchain-community)
  Using cached aiohttp-3.12.15-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (7.7 kB)
Requirement already satisfied: tenacity<=8.4.0,<10,>=8.1.0 in ./venv/lib/python3.12/site-packages (from langchain-community) (8.4.0)
Collecting dataclasses-json<0.7,>=0.5.7 (from langchain-community)
```

Note: If prompted, select the Python interpreter from the newly created virtual environment. If a pop-up asks to install ipykernel, click **Install** and re-run the first cell.

1.4 Import the required libraries using the following code:

```
import os
from langchain_openai import AzureChatOpenAI
from langchain_core.prompts import PromptTemplate
```

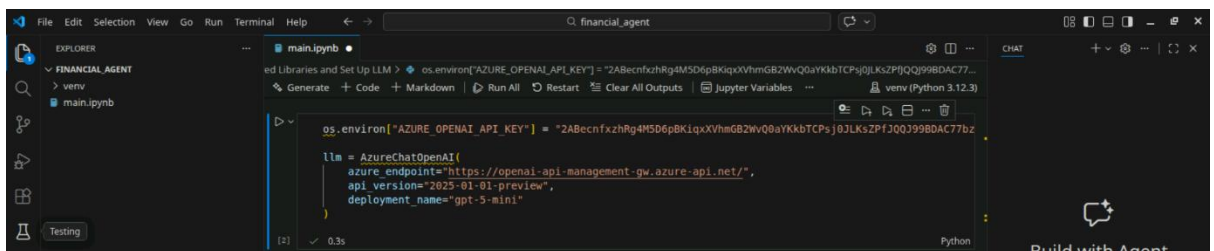


Step 2: Configure the Azure OpenAI model

2.1 Set your credentials and initialize the model using the following code:

```
os.environ["AZURE_OPENAI_API_KEY"] =
"2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77b
zfXJ3w3AAABACOGi3sC"
```

```
llm = AzureChatOpenAI(
    azure_endpoint="https://openai-api-management-gw.azure-api.net/",
    api_version="2025-01-01-preview",
    deployment_name="gpt-5-mini"
)
```



Note: This configures the GPT-5 model for deterministic outputs using the Azure OpenAI API credentials.

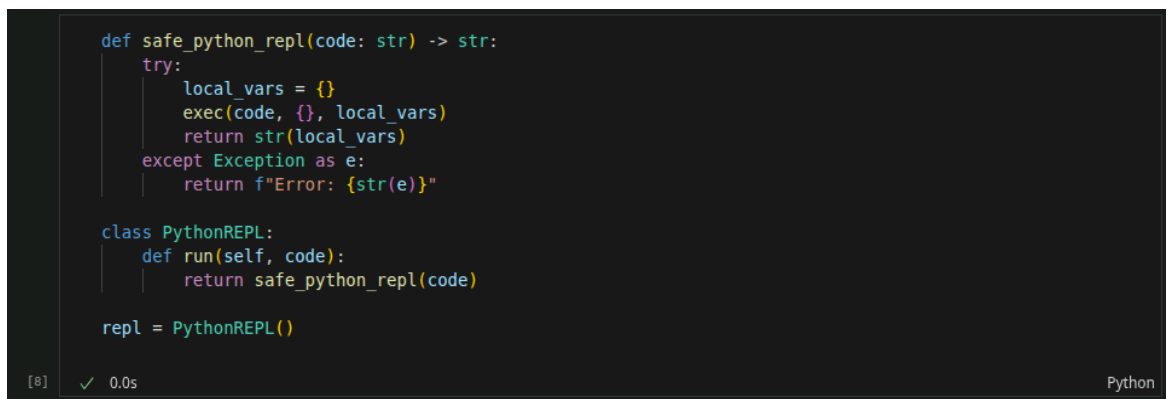
Step 3: Define the execution tool and prompt template

3.1 Define a custom Python execution tool using the following code:

```
def safe_python_repl(code: str) -> str:
    try:
        local_vars = {}
        exec(code, {}, local_vars)
        return str(local_vars)
    except Exception as e:
        return f"Error: {str(e)}"

class PythonREPL:
    def run(self, code):
        return safe_python_repl(code)

repl = PythonREPL()
```

A screenshot of a code editor with a dark background. The code is written in Python and is color-coded. It defines a function 'safe_python_repl' that takes a string 'code' and returns a string. Inside the function, there is a try-except block. The try block initializes an empty dictionary 'local_vars', executes the code in that context using 'exec', and returns the string representation of 'local_vars'. The except block catches any 'Exception' and returns a formatted error message. Below the function, a class 'PythonREPL' is defined with a 'run' method that calls 'safe_python_repl'. Finally, an instance 'repl' of 'PythonREPL' is created. At the bottom left, there is a status bar showing '[0]' and a green checkmark followed by '0.0s'. At the bottom right, the word 'Python' is visible.

```
def safe_python_repl(code: str) -> str:
    try:
        local_vars = {}
        exec(code, {}, local_vars)
        return str(local_vars)
    except Exception as e:
        return f"Error: {str(e)}"

class PythonREPL:
    def run(self, code):
        return safe_python_repl(code)

repl = PythonREPL()
```

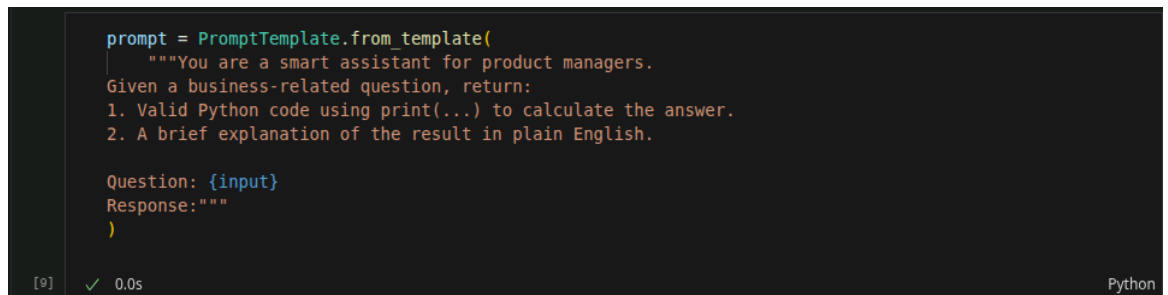
[0] ✓ 0.0s Python

Note: The REPL (Read-Eval-Print Loop) lets you run Python code interactively as you work on a project or learn the language. It comes with every Python installation, so you can use it anytime.

3.2 Use the following code with a structured prompt to get Python code for the calculation and a brief explanation in plain English:

```
prompt = PromptTemplate.from_template(
    """You are a smart assistant for product managers.
    Given a business-related question, return:
    1. Valid Python code using print(...) to calculate the answer.
    2. A brief explanation of the result in plain English.

    Question: {input}
    Response: """
)
```

A screenshot of a code editor with a dark background. The code is written in a light-colored font. It shows the same prompt template code as in the previous block. At the bottom left, there is a status bar with a green checkmark, a small icon, and the text '0.0s'. At the bottom right, the word 'Python' is visible.

```
prompt = PromptTemplate.from_template(
    """You are a smart assistant for product managers.
    Given a business-related question, return:
    1. Valid Python code using print(...) to calculate the answer.
    2. A brief explanation of the result in plain English.

    Question: {input}
    Response: """
)
```

Note: This prompt instructs the model to return both code and a plain-English explanation in a consistent format.

Step 4: Define the processing logic and business questions

4.1 Define a function to process the question, extract the code, and execute it using the following code:

```
def process_pm_query(q):
    response = llm.invoke(prompt.format(input=q)).content.strip()

    if "`python" in response:
        code_block = response.split("`python")[1].split("`")[0].strip()
    else:
        code_block = next((line for line in response.splitlines() if "print" in line), "")

    try:
        result = repl.run(code_block)
        return {
            "question": q,
            "code": code_block,
            "result": result.strip(),
            "explanation": response.replace(code_block, "").replace("`python",
            "").replace("`", "").strip()
        }
```

```

    }
except Exception as e:
    return {
        "question": q,
        "code": code_block,
        "error": str(e),
        "raw_response": response
    }

```

```

def process_pm_query(q):
    response = llm.invoke(prompt.format(input=q)).content.strip()

    if "`python" in response:
        code_block = response.split("`python")[1].split("`")[0].strip()
    else:
        code_block = next((line for line in response.splitlines() if "print" in line), "")

    try:
        result = repl.run(code_block)
        return {
            "question": q,
            "code": code_block,
            "result": result.strip(),
            "explanation": response.replace(code_block, "").replace("`python", "").replace("`", "").strip()
        }
    except Exception as e:
        return {
            "question": q,
            "code": code_block,
            "error": str(e),
            "raw_response": response
        }

```

Note: This function sends the question to the model, extracts code from the response, executes it using Python REPL, and returns both the code and the explanation.

4.2 Define sample business questions using the following code:

```

questions = [
    "If we grow 8% monthly, what is our user count after 6 months starting from 10,000?",
    "We lose 25% of our 40,000 users. How many do we retain?",
    "If each user pays $10/month and we have 5000 users, what is the monthly revenue?",
    "If revenue is $50,000 and cost is $37,000, what's our profit?"
]

```

```

questions = [
    "If we grow 8% monthly, what is our user count after 6 months starting from 10,000?",
    "We lose 25% of our 40,000 users. How many do we retain?",
    "If each user pays $10/month and we have 5000 users, what is the monthly revenue?",
    "If revenue is $50,000 and cost is $37,000, what's our profit?"
]

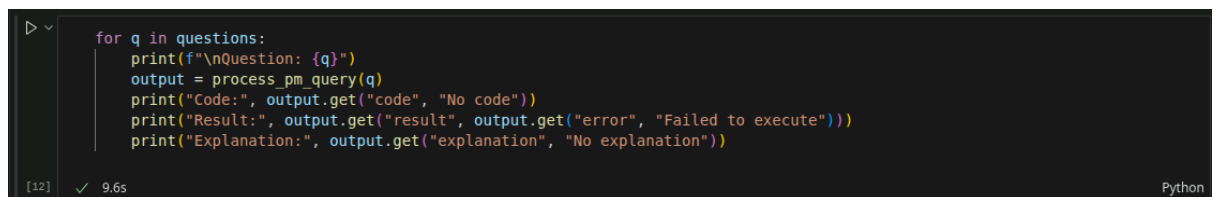
```

Note: These test cases reflect common business scenarios that product managers face.

Step 5: Run the agent loop and view the outputs

5.1 Execute the agent loop and view the outputs using the following code:

```
for q in questions:
    print(f"\nQuestion: {q}")
    output = process_pm_query(q)
    print("Code:", output.get("code", "No code"))
    print("Result:", output.get("result", output.get("error", "Failed to execute")))
    print("Explanation:", output.get("explanation", "No explanation"))
```

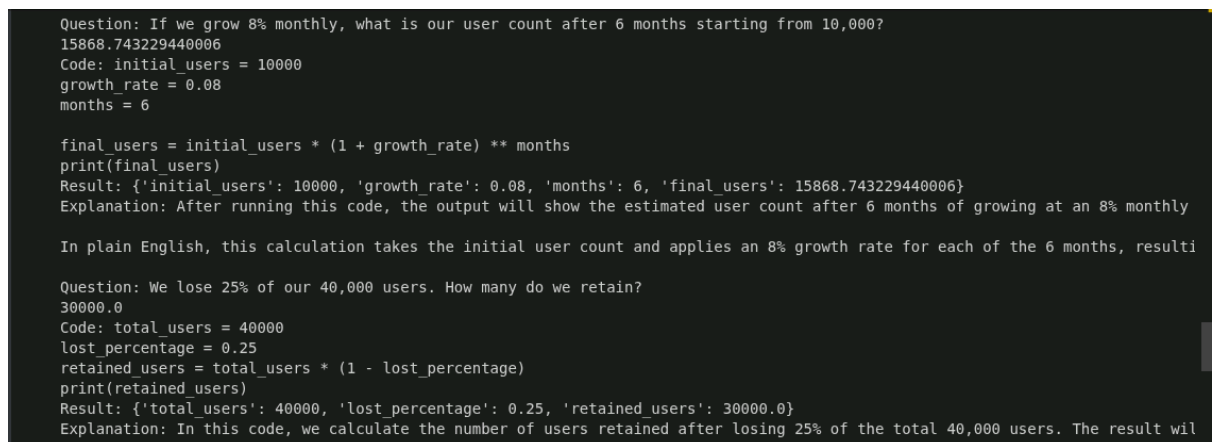


```
for q in questions:
    print(f"\nQuestion: {q}")
    output = process_pm_query(q)
    print("Code:", output.get("code", "No code"))
    print("Result:", output.get("result", output.get("error", "Failed to execute")))
    print("Explanation:", output.get("explanation", "No explanation"))
```

[12] ✓ 9.6s Python

Note: This loop processes each question, runs the code, and displays both the output and the explanation for quick business insights.

The output appears as shown in the following screenshot:



```
Question: If we grow 8% monthly, what is our user count after 6 months starting from 10,000?
15868.743229440006
Code: initial_users = 10000
growth_rate = 0.08
months = 6

final_users = initial_users * (1 + growth_rate) ** months
print(final_users)
Result: {'initial_users': 10000, 'growth_rate': 0.08, 'months': 6, 'final_users': 15868.743229440006}
Explanation: After running this code, the output will show the estimated user count after 6 months of growing at an 8% monthly

In plain English, this calculation takes the initial user count and applies an 8% growth rate for each of the 6 months, resulti

Question: We lose 25% of our 40,000 users. How many do we retain?
30000.0
Code: total_users = 40000
lost_percentage = 0.25
retained_users = total_users * (1 - lost_percentage)
print(retained_users)
Result: {'total_users': 40000, 'lost_percentage': 0.25, 'retained_users': 30000.0}
Explanation: In this code, we calculate the number of users retained after losing 25% of the total 40,000 users. The result wil
```

By following these steps, you will create a LangChain financial Q&A agent that turns business questions into executable calculations and clear explanations, enabling faster decisions while reducing spreadsheet work and mistakes.